The method `multiline?` says whether a regexp has the `/m` flag set, that is, whether the dot matches newlines.

```
%r{.}.multiline?  # => false
%r{.}m.multiline? # => true

Regexp.new('.').multiline?                    # => false
Regexp.new('.', Regexp::MULTILINE).multiline? # => true
```

Rails uses this method in a single place, also in the routing code. Multiline regexps are disallowed for route requirements and this flag eases enforcing that constraint.

```
def assign_route_options(segments, defaults, requirements)
  ...
  if requirement.multiline?
    raise ArgumentError, "Regexp multiline option not allowed in
routing requirements: #{requirement.inspect}"
  end
  ...
end
```

Defined in `active_support/core_ext/regexp.rb`.

13 Extensions to Range

13.1 to_s

Active Support extends the method `Range#to_s` so that it understands an optional format argument. As of this writing the only supported non-default format is `:db`:

```
(Date.today..Date.tomorrow).to_s
# => "2009-10-25..2009-10-26"

(Date.today..Date.tomorrow).to_s(:db)
# => "BETWEEN '2009-10-25' AND '2009-10-26'"
```

As the example depicts, the `:db` format generates a `BETWEEN` SQL clause. That is used by Active Record in its support for range values in conditions.

Defined in `active_support/core_ext/range/conversions.rb`.

13.2 include?

The methods `Range#include?` and `Range#===` say whether some value falls between the ends of a given instance:

```
(2..3).include?(Math::E) # => true
```

Active Support extends these methods so that the argument may be another range in turn. In that case we test whether the ends of the argument range belong to the receiver themselves:

```
(1..10).include?(3..7)  # => true
(1..10).include?(0..7)  # => false
(1..10).include?(3..11) # => false
(1...9).include?(3..9)  # => false

(1..10) === (3..7)  # => true
(1..10) === (0..7)  # => false
(1..10) === (3..11) # => false
(1...9) === (3..9)  # => false
```

Defined in `active_support/core_ext/range/include_range.rb`.

13.3 overlaps?

The method `Range#overlaps?` says whether any two given ranges have non-void intersection:

```
(1..10).overlaps?(7..11)  # => true
(1..10).overlaps?(0..7)   # => true
(1..10).overlaps?(11..27) # => false
```